BloodDrop

Questions you may be asked, and how to answer them — by layer

Grouped into frontend, backend, and database. The harder questions are the ones about why a decision was made rather than what the code does, so those are answered first in each section.

Frontend — React, Vite, Tailwind, Leaflet

Why React with plain JavaScript rather than TypeScript?

The project is small enough that the type safety would not have paid for the setup cost, and the risky logic — compatibility, eligibility, distance — lives on the server where it is covered by tests. On a larger team I would use TypeScript.

How does the map work, and what does it cost?

Leaflet with OpenStreetMap tiles. Nothing: no API key, no billing, no usage cap. Google Maps would need a billing account. Markers are drawn as div icons rather than images, so there are no broken asset paths after bundling.

Why is blood group a grid of buttons instead of a dropdown?

There are only eight values and it is the field every screen turns on. A grid is one tap instead of tap–scroll–tap, which matters on a phone in an emergency. It is built on real radio inputs, so keyboard and screen-reader behaviour comes for free.

How does dark mode work?

A single `dark` class on the root element. Tailwind compiles every `dark:` utility into a rule scoped to it, so both rule sets ship and one class decides which applies — the switch costs no network request and no re-render. An inline script in index.html sets the class before React mounts, otherwise a dark-mode user gets a white flash on every load.

How do you keep the response board live?

Polling, paused while the tab is hidden and refetching when it returns, with a minimum gap so flicking between tabs cannot cause a burst. This mattered: the first version polled regardless of visibility and one open tab made roughly ten thousand API calls a day.

Why not Supabase Realtime instead of polling?

Polling works identically in the offline mock and against Supabase, which keeps one code path. Realtime would be the upgrade, and it would only change the hook.

How is state managed?

React context for authentication and theme; everything else is local component state. There is no Redux because there is no shared mutable state complex enough to need it.

What stops a donor's exact home location being shown?

Coordinates are rounded to roughly a kilometre grid before they reach anyone else, and only a distance and an area label are displayed.

Backend — Node, Express

Where does the domain logic live, and why not in the routes?

In small single-purpose modules under `lib/` — compatibility, eligibility, distance, ranking, masking, slots. Route handlers validate input and call them. It means the rules can be read without untangling HTTP, and tested directly.

Explain the matching algorithm.

Given a patient's group, derive the donor groups that can safely give to it. Ask the database for donors of those groups inside the radius. Drop anyone inside the 90-day gap or who has paused. Rank by distance with a small reliability adjustment, so a nearer donor always wins but a reliable one breaks a tie. Alert a capped shortlist rather than everyone.

Why cap the number alerted?

Alerting everyone in range trains people to ignore alerts. A capped, ranked shortlist keeps them meaningful, and the requester can send another round that reaches only donors not already contacted.

How does phone masking actually work?

Numbers never leave the server in list or search responses — they are masked before serialisation. They unlock only between a donor who accepted and the requester of that request, through a dedicated endpoint that also notifies the person whose number was shown. There is also a relay so both sides can message with no number shared.

Why does the blood bank confirm donations rather than the requester?

Because the bank is the only party that witnessed it. When the requester confirmed, they could credit a friend or mark a genuine donor absent, which made the reliability score self-reported by someone with an interest in it. A requester who tries now receives a 409 pointing at the bank.

How are the bookable slots produced?

Derived at request time from the facility's opening hours, slot length and capacity — not stored. A facility changing its hours therefore needs no migration and no backfill; only the bookings are rows. Past slots are filtered out, and a slot at capacity is marked full.

What happens to an uploaded document?

It is size-checked and type-checked on the server rather than trusting the browser, stored privately, shown to one moderator, and deleted the moment a decision is recorded. Requesting it afterwards returns 410. Only the decision and the kind of evidence survive.

How do you avoid N+1 queries?

Collections are fetched with a single `in (…)` lookup and joined in memory. A 25-donor response board was 50 sequential round trips before this; it is now two.

What stops a deployment running against the wrong database?

If the app detects a deployed environment with no Supabase credentials, it refuses to start and names the missing variable, rather than falling back to the in-memory database — which would look healthy while discarding every registration on each cold start.

How is it tested?

A single suite of 100 assertions boots the real Express app on an ephemeral port and drives it over HTTP, including the negative cases. It runs against the in-memory database, so it needs no credentials and no network.

Database — Supabase, Postgres, PostGIS

Why PostGIS rather than calculating distance in the application?

Because filtering has to happen before the rows are fetched. `ST_DWithin` uses a GiST index on a geography column, so the database returns only donors inside the radius. Pulling every donor and measuring in JavaScript would work for a hundred donors and collapse at a hundred thousand.

What is the geography column and why is it generated?

A `geography(Point, 4326)` column derived automatically from the lat and lng the app writes. Generating it means the spatial data can never drift out of step with the coordinates, which it would if the application had to remember to update both.

Why is distance for facilities computed in JavaScript instead?

A region has dozens of facilities but potentially thousands of donors. An indexed spatial query earns its complexity for one and not the other — that is a deliberate asymmetry, not an inconsistency.

What does row-level security give you here?

15 policies ensure that even with direct database access a user reads only their own profile, donor record, responses and notifications. The API uses the service role and bypasses them, so they are defence in depth rather than the primary control — demonstrated by the public key returning zero donor rows.

Why does the app hold its own counters instead of counting rows?

Reliability needs alerts received, responses given, donations confirmed and no-shows for every donor in a match set. Counting rows per donor on each match would be the N+1 problem again. The counters are written at the same points the events occur.

How is the schema managed?

3 ordered SQL migrations under `supabase/migrations/`. One lesson learned: adding a column to a function's return type requires dropping the function first, since Postgres will not replace one whose row type changed — so that migration drops and recreates inside a transaction.

Where are uploaded files stored?

A private Supabase Storage bucket, size- and type-restricted, never public. Locally the same interface writes to disk instead, so the feature works with no account.

What happens when the free Supabase project pauses?

The hostname stops resolving and the browser reports a failed fetch at login. It is a property of the free tier after about a week of inactivity, and the project is restored from the dashboard with data and keys intact.

Questions about judgement

What is the hardest problem in this project?

Not the geography — it is deciding who is allowed to assert what. Blood group, whether a donation happened, and whether a request is genuine are all claims someone benefits from making falsely. Most of the design is about moving each claim to the party with no incentive to lie: the blood bank confirms donations, a moderator confirms blood groups and hospital badges, and the donor alone decides when their number is revealed.

What would you build next?

SMS, because the people least likely to have the app open are the ones in the areas with the thinnest donor coverage. After that, predictive shortage alerts from request history.

What is the weakest part?

Moderation does not scale. One queue worked by hand is fine for a demonstration and would fail immediately at volume; it needs rate limits, duplicate detection and trusted-reporter weighting.

What did you get wrong and fix?

The first design had donors meeting requesters directly, which is not how transfusion works — blood must be collected at a licensed facility. Adding facilities also exposed a worse flaw: the requester was confirming donations, so the reliability score was being reported by the person it benefited.